江西财经大學
JIANGXI UNIVERSITY OF FINANCE AND ECONOMICS

普通本科毕业论文（设计）

开 题 报 告 书

学　　院：	School of Computing and Artificial Intelligence
专　　业：	Computer Science
学　　号：	2202230024
学生姓名：	OUSSAMA ESSABTI
指导老师：	Lai Xinfeng
完成日期：	2026 年 01 月 10 日

论文题目	Design and Implementation of a Web Dashboard for Kubernetes Cluster Monitoring and Management	
论文形式	**文科类专业**可采用的形式： ☐毕业论文☐学术论文☐案例分析☐调研报告☐创作方案设计☐项目设计☐模拟审判卷宗☐其他 **理工类及交叉类专业**可采用的形式： ■毕业设计☐毕业创作☐学术论文☐调查报告☐毕业作品（原型）☐发明☐专利☐其他	
选题来源	■是 ☐否	是否属于以实验、实习、工程实践和社会调查等实践性工作为基础的毕业论文（设计）。

研究背景与研究意义

1. Research Objectives:

The main goal of this research is to develop a smart, web-based dashboard for Kubernetes clusters. This system is designed to turn complex data into clear, easy-to-understand information. The project focuses on creating a single platform that combines metrics, logs, and events in one place. Additionally, it aims to use the "Isolation Forest" algorithm to find system problems automatically and create a simple visual layout that helps engineers manage clusters without feeling overwhelmed.

2. Social, Technical, and Academic Background:

From a social perspective, our modern world relies on cloud services for everything from banking to healthcare. When these systems fail, it affects many people's daily lives. **Technically**, as more companies move to "microservices," the number of containers has exploded. A 2024 survey shows that 96% of organizations now use Kubernetes, making it the industry standard. **Academically**, current research in "AIOps" (Artificial Intelligence for IT Operations) shows that while we have plenty of data, we lack tools that can analyze it all at once to find the real cause of a problem quickly.

3. Motivation and Problem Solving:

The motivation for this project comes from the fact that current tools are often scattered and hard to use. Engineers often have to jump between different programs to see what is wrong, which wastes time during a crisis. This project solves these problems by:

- **Unifying Data**: Bringing everything (logs and metrics) into one screen.
- **Reducing "Alert Fatigue"**: Instead of sending hundreds of confusing alerts, the system uses the **Isolation Forest** algorithm to pick out only the real, important issues.
- **Practicality**: The dashboard is built to be user-friendly, allowing even small teams to manage complex clusters effectively.

4. Technical Progressiveness and Innovation:

The progressiveness of this project lies in its transition from passive monitoring to active, intelligent observation. Unlike traditional tools that merely display raw data, this system implements a "smart" layer to interpret cluster health.

- **Algorithmic Innovation**: By integrating **Unsupervised Machine Learning (Isolation Forest)**, the system eliminates the need for manual threshold configuration. This is a significant step forward as the system autonomously learns the baseline of "normal" operations and identifies anomalies that human operators might miss, making the monitoring process proactive rather than reactive.
- **Full-Stack Technological Advancement**: The architecture leverages a modern stack consisting of **FastAPI (Asynchronous Backend)** and **React (Frontend)**. The core innovation here is the use of **WebSockets** for bi-directional, real-time communication. This ensures "Zero-Latency Observation," meaning the state of the Kubernetes cluster is reflected on the user's screen instantly, which is critical for high-stakes production environments.
- **Scalable Practical Application**: The system is designed with a **Microservices-based Modular Architecture**. This means the dashboard is not a static tool but a scalable framework. As a cluster expands from a few nodes to hundreds, the system's data collection and analysis modules can scale independently. This makes it a highly practical and future-proof solution for modern DevOps teams managing dynamic, cloud-native infrastructures.

文献综述（要求：1. 所用的文献都应和论文研究问题直接相关，应特别注意对主流、权威文献学术成果的引用和评述。2. 可以按文献与论文主题的关系由远而近进行综述，也可以按年代顺序综述，也可按不同的问题进行综述，还可按不同的观点进行比较综述。3. 末尾按规范格式列出所用的参考文献）

I. Research Objectives and Background

1. Social Background

With the rapid development and popularization of cloud computing technology, containerization technology represented by Kubernetes has become the core foundation of modern IT infrastructure. Kubernetes has been extensively adopted in enterprise-level applications, providing robust mechanisms for container orchestration, cluster management, and microservices architecture support, offering an efficient and flexible solution for large-scale distributed systems. As Burns et al. [1] emphasize in "Borg, Omega, and Kubernetes: Lessons Learned from Three Container-Management Systems Over a Decade," container orchestration systems like Kubernetes require sophisticated management capabilities to handle the complexities of distributed environments effectively. Wang et al. [13] provide comprehensive introduction to cloud computing development history, technical characteristics, and industry applications, establishing the foundational context for modern containerization technologies.

However, as system scale continues to expand and the business environment becomes increasingly complex, container environments face unprecedented challenges in monitoring and troubleshooting. Traditional monitoring methods often fail to promptly detect and resolve system failures, leading to service interruptions and performance degradation. Sharma et al. [2] point out that effective monitoring in containerized environments requires comprehensive data collection and analysis capabilities that span multiple layers of the infrastructure, from host-level system metrics to application-level performance indicators. The complexity multiplies when dealing with orchestrated container environments where services are dynamically scheduled, moved, and scaled based on resource availability and demand patterns.

The core issue lies in the fact that existing monitoring tools often struggle to effectively visualize and analyze large volumes of data, making it difficult for operations personnel to gain timely insights into system health and failure root causes. Recent research by Jaramillo et al. [3] on Docker monitoring and performance challenges highlights the need for real-time resource monitoring and intelligent anomaly detection capabilities specifically designed for containerized environments. As containers become increasingly ephemeral and microservices architectures more distributed, the traditional approach of setting static thresholds and alerts becomes inadequate for capturing the dynamic nature of modern cloud-native systems.

2. Technical Background

In modern IT operations, the importance of monitoring and troubleshooting is self-evident. Effective monitoring can promptly identify potential issues, reducing the impact of failures; efficient troubleshooting can quickly locate problem root causes, shortening system recovery time. Malygin [4] comprehensively analyzes microservice architecture risks and opportunities in cloud systems from 2024-2030, emphasizing that modern cloud environments require sophisticated monitoring and observability frameworks to handle architectural complexity.

International and Chinese researchers have extensively studied cloud computing infrastructure. Zhou [14] explores the translation and practical application of cloud computing concepts in Chinese contexts, while Wu [16] discusses development trends of cloud computing virtualization technology, including container and Docker technologies that underpin modern Kubernetes deployments. In the context of Kubernetes, the distributed and dynamic nature of systems makes monitoring and troubleshooting significantly more complex, demanding a more intelligent and automated approach.

Existing Kubernetes monitoring solutions, such as Prometheus and Grafana, while capable of collecting and displaying various metrics, often lack sufficient intelligence in anomaly detection and root cause analysis capabilities. Traditional rule-based alert mechanisms frequently generate numerous false positives or false negatives, increasing operational burden and potentially missing critical failures. As described by Artac et al. [5], who studied DevOps infrastructure-as-code automation, the complexity of modern cloud-native applications necessitates more sophisticated monitoring approaches that go beyond simple metric collection. The challenge intensifies when considering that Kubernetes environments often involve hundreds or thousands of containers distributed across multiple nodes, with complex networking, storage, and service interdependencies that create intricate failure propagation patterns.

The limitations of existing technologies primarily manifest in three critical aspects: first, lack of sufficient context awareness in data visualization, making it difficult for operations personnel to understand the relationships between different system components and how failures in one service impact the broader system; second, weak anomaly detection capabilities, unable to automatically identify and predict potential failures before they impact users; third, inefficient root cause analysis, requiring operations personnel to invest significant time and effort manually correlating logs, metrics, and traces to pinpoint the underlying causes of system failures.

3. Academic Significance

To address these issues, this research aims to develop a Kubernetes monitoring and troubleshooting dashboard integrated with intelligent anomaly detection and automated root cause analysis capabilities, achieving "observable clustering" and "automatic root cause analysis." This research carries important theoretical and practical value, reflected in the following aspects:

Theoretically, this research explores the application of machine learning algorithms in anomaly detection and root cause analysis, particularly in handling complex metric relationships and time-series data. As demonstrated by Nedelkoski et al. [6] in their study on self-attentive classification-based anomaly detection, advanced machine learning techniques can significantly improve the accuracy of anomaly detection in cloud systems. The project will investigate how to automatically extract anomaly patterns and identify abnormal behavior without manual rule definition. Additionally, by analyzing container logs with machine learning models, the project will examine how to enhance the efficiency of root cause location.

Practically, this research will provide a highly usable, intelligent monitoring tool that significantly improves the efficiency of Kubernetes cluster operations and reduces failure impact. The system will enable operations personnel to more intuitively understand system states, promptly discover potential issues, and quickly locate failure root causes. Research by Cerny et al. [7] on contextual understanding of microservice architecture supports the need for

comprehensive monitoring solutions. Furthermore, the research will demonstrate the extensive application potential of machine learning in IT operations, providing reference cases for the development and implementation of intelligent operations (AIOps). Xu et al. [8] propose Deep Isolation Forest, which significantly advances anomaly detection by combining deep learning with isolation-based methods, offering robust theoretical foundations for the project's anomaly detection implementation in high-dimensional Kubernetes monitoring data.

II. Research Status

1. Observability and Monitoring Technologies:

Observability is the ability to infer a system's internal state by examining its external outputs. In the realm of cloud-native applications, observability typically relies on three pillars: metrics, logs, and traces. Research on observability foundations provides comprehensive frameworks for understanding system behavior. Metrics quantify system behavior (e.g., CPU, memory, network traffic), logs record event details, and traces track request flows, collectively enabling comprehensive system monitoring.

Kubernetes monitoring primarily focuses on collecting and analyzing various metrics. Key monitoring objects include cluster-level metrics (such as node status, resource utilization, cluster capacity), namespace-level metrics (such as pod count, resource quotas, namespace isolation), and application-level metrics (such as container performance, application-specific business metrics). Common monitoring tools include Prometheus, an open-source monitoring system that uses a time-series database to store metrics data with excellent scalability properties and supports flexible querying and alerting through PromQL; and Grafana, a visualization platform for data from diverse sources, creating rich, customizable dashboards that provide intuitive representations of system state. These tools have become industry standards for cloud-native monitoring. Research by Villamizar et al. [9 demonstrates that microservice architectures provide better scalability and cost efficiency in cloud environments, which is critical for understanding the benefits of adopting Kubernetes for modern applications and the associated monitoring requirements that arise from distributed architectural patterns.

While these tools are highly powerful and feature-rich, they typically require significant configuration and tuning to be effective, and fundamentally lack intelligent analysis capabilities. Moreover, the vast volume of metrics data often overwhelms operations personnel, leading to "alert fatigue"—a phenomenon where the sheer number of alerts desensitizes operators to actual critical issues. Additionally, the static nature of alert thresholds means they cannot adapt to changing application behavior patterns over time.

2. Anomaly Detection Technologies:

Anomaly detection aims to identify patterns that deviate from expected behavior. In IT operations, anomaly detection can promptly discover system failures, performance degradation, or security incidents before they significantly impact service availability. Traditional anomaly detection methods primarily rely on manually defined rules or thresholds, such as triggering alerts when CPU utilization exceeds 80% or memory usage surpasses 90%. However, these methods face critical issues of high false positive rates (generating alerts for benign conditions) and

difficulty adapting to complex dynamic environments where normal operating ranges vary based on workload type, time of day, and business context.

In recent years, machine learning has been extensively applied to anomaly detection to overcome these limitations. Supervised learning methods require labeled historical data to train models that can distinguish between normal and abnormal behavior. However, abnormal samples are often scarce in practice, making supervised learning difficult to implement effectively and economically. Unsupervised learning methods, such as clustering and dimensionality reduction techniques, automatically discover abnormal patterns without labeled data, making them more practical for operational environments. Effective visualization of anomaly detection results is essential for operational insights, as evidenced by research from Samadbeik et al. [10] on dashboard technologies for complex system monitoring and decision support. Advanced anomaly detection applications extend beyond traditional IT operations; Song [20] develops anomaly target detection methods in hyperspectral remote sensing using isolation forest techniques, demonstrating the broad applicability of these algorithms across different domains and validating their effectiveness in detecting subtle anomalies in high-dimensional datasets.

In the context of Kubernetes monitoring, anomaly detection must address several distinctive challenges: first, handling high-dimensional time-series data, as each container typically generates dozens of metrics simultaneously, creating complex feature spaces; second, capturing complex metric relationships and dependencies, as failures may result not from individual metric deviations but from unusual combinations of metric interactions; third, adapting to dynamic environments, as container lifecycles are short and system topology changes frequently with deployments and scaling events; fourth, maintaining computational efficiency to enable real-time detection without introducing unacceptable latency into the monitoring pipeline.

3. Root Cause Analysis Technologies

Root cause analysis seeks to identify the underlying causes of failures or anomalies through systematic investigation and correlation of evidence. In complex distributed systems like Kubernetes, locating root causes is highly challenging, as failures may result from hardware issues (node failures, network problems), software bugs (application code defects, library incompatibilities), configuration errors (incorrect resource limits, misconfigured networking), or external dependencies (database unavailability, third-party API failures). Traditional root cause analysis primarily relies on manual investigation by operations personnel, reviewing logs, metrics, and traces to identify problems—a time-consuming, labor-intensive process prone to human error and subject to investigator bias. Dragoni et al. [11] provide comprehensive coverage of microservices architecture principles that inform distributed system troubleshooting and help establish the foundational understanding necessary for effective root cause analysis.

Automated root cause analysis technologies aim to reduce or eliminate manual intervention and accelerate the resolution process. Common approaches include rule-based systems (which encode domain expert knowledge), case-based reasoning (which leverage historical failure patterns), and machine learning methods (which learn complex relationships from data). Rule-based systems define a series of rules based on domain knowledge to map failure symptoms to potential causes; however, maintaining these rules is costly, difficult, and often incomplete. Case-based reasoning leverages historical failure cases to identify similar situations, but its effectiveness depends heavily on case database quality and completeness. Machine learning

methods learn root cause identification patterns from historical data, offering strong generalization capabilities; nevertheless, they require substantial labeled data and lack the interpretability needed for operational trust. Hasselbring & Steinacker [12] explore microservice architecture patterns that are foundational to understanding failure modes and their propagation in modern distributed systems. Wang et al. [15] review access control technology research in cloud computing environments, providing security perspectives on system resilience and protection mechanisms. Zhang et al. [18] review trusted cloud computing research and security technologies, addressing critical aspects of secure and reliable system operation in untrusted environments. Jiang et al. [17] review current status and applications of cloud computing technology, contributing to broader understanding of cloud system reliability and best practices.

4. Research Status Summary

In summary, while significant progress has been made in Kubernetes monitoring, anomaly detection, and root cause analysis, critical gaps remain:

1. Integration Issues: Existing monitoring tools, anomaly detection algorithms, and root cause analysis methods are often developed independently, lacking effective integration. Operations personnel need to switch between multiple tools and platforms, reducing efficiency.

2. Limited Intelligence: Most monitoring tools still rely on manually configured rules and thresholds, lacking automated learning and adaptive capabilities, unable to effectively handle increasingly complex dynamic environments.

3. Poor User Experience: Existing tools often have overly complex interfaces and operations, presenting too much raw data without providing intuitive, actionable insights. Operations personnel need to invest substantial time and effort to interpret data and locate issues.

III. Current Development Status

To address the above issues, this research proposes developing a Kubernetes monitoring and troubleshooting dashboard with intelligent anomaly detection and automated root cause analysis capabilities. The project has made substantial progress in preliminary design and development, with the following status overview:

1. Architecture Design:

The project adopts microservices architecture, with frontend and backend separation. The backend leverages Kubernetes API to collect cluster metrics and logs, utilizing machine learning algorithms for anomaly detection and root cause analysis; the frontend employs modern JavaScript frameworks to provide intuitive, user-friendly interfaces.

Specifically, the backend includes four core modules: the Data Collection Module is responsible for extracting metrics, logs, and events from Kubernetes API; the Anomaly Detection Module applies machine learning algorithms (primarily Isolation Forest) to identify abnormal metrics or logs; the Root Cause Analysis Module associates anomaly detection results with logs and events to locate potential root causes; the API Service Module provides RESTful APIs for frontend data retrieval. The frontend comprises three core modules that directly shape the dashboard experience: the Cluster Overview surface offers aggregated cluster health, node conditions, namespace quotas, and service states; the Resource Monitoring views provide drill-

down CPU/memory/storage/network timelines with selectable time ranges; the Anomaly Detection panel visualizes Isolation Forest scores, outlier flags, and confidence bands; and the Root Cause Analysis panel threads correlated alerts, logs, and events into an interactive timeline with linked detail views. Each module is designed for clarity, fast scanning, and rapid drill-down from overview to granular pod-level evidence.

2. Backend Development:

Backend development focuses on implementing data collection, anomaly detection, and root cause analysis functionalities. The Data Collection Module connects to the Kubernetes API server to enumerate nodes, pods, and services, and queries Prometheus for time-series metrics; it also streams Kubernetes events and pod logs to provide context. The Anomaly Detection Module has integrated the Isolation Forest algorithm for detecting abnormal metrics. Isolation Forest is an unsupervised learning algorithm that identifies anomalies by randomly constructing decision trees; anomalies, being sparse and having distinct characteristics, are isolated in shallower tree layers. The algorithm is simple and efficient, well-suited for high-dimensional data and large-scale datasets. The Root Cause Analysis Module links anomaly spikes with contemporaneous events and log excerpts, producing a ranked list of suspected causes with associated evidence (timestamped events, log snippets, involved pods/nodes). The API layer exposes RESTful endpoints (e.g., /api/cluster/overview, /api/metrics/{resource}, /api/anomalies, /api/rca/{incident}) returning JSON payloads optimized for the dashboard, with pagination and time-range parameters to support incremental loading.

3. Frontend Development:

Frontend development emphasizes providing intuitive, user-friendly interfaces. The Cluster Overview page presents health summaries with status badges, sparklines for recent resource trends, and topology-aware node/service tables. The Resource Monitoring views render linked time-series charts (CPU, memory, storage I/O, network) with brush-zoom, tooltips, threshold bands, and tabular rollups; users can filter by namespace, workload, or node and pin charts for side-by-side comparisons.

The Anomaly Detection panel overlays Isolation Forest scores and flags on metric timelines, supports sorting by severity, and allows interactive what-happened-at-this-time exploration. The Root Cause Analysis panel threads anomalies with events and logs on a synchronized timeline, enabling drill-down to log snippets and involved pods. As detailed by Samadbeik et al. [10], effective dashboard technologies and data visualization designs are essential for supporting complex decision-making and system monitoring tasks. The frontend uses responsive layouts, skeleton loading states, and incremental data requests to maintain smooth interactivity while presenting dense operational data, improving operational efficiency and reducing mean-time-to-resolution (MTTR).

4. Data Freshness and Real-time Updates:

Ensuring data freshness is critical for effective monitoring. The project plans to implement mechanisms ensuring metrics and logs remain current. For metrics, the backend will periodically poll Kubernetes API and Prometheus to retrieve the latest data (e.g., every 30 seconds for overviews, faster for critical signals). For logs, the backend will leverage Kubernetes log

streaming API to obtain real-time log updates. The frontend will use WebSocket or Server-Sent Events (SSE) to receive real-time data updates from the backend, ensuring dashboard content is always up-to-date. Additionally, the project will implement caching mechanisms to reduce unnecessary API calls and improve performance.

To handle high-dimensional and large-scale data efficiently, the system will batch metric queries, parallelize per-namespace or per-node requests, and down-sample time-series for visualization while preserving anomaly-relevant features. Rate limiting and backpressure will be applied on streaming channels to prevent UI overload. Incremental updates (diffs) will be pushed instead of full datasets to reduce payload size; the frontend will throttle chart refreshes and coalesce updates to avoid repaint storms.

However, real-time updates introduce challenges. First, high-frequency data updates may increase backend load and network bandwidth consumption; optimizing data collection frequency and transmission efficiency is necessary.

Second, excessive frontend update frequency may cause interface flickering or performance degradation; the frontend needs to implement appropriate throttling or buffering mechanisms to ensure smooth user experience. Third, real-time data processing and analysis demand high computational resources; backend algorithms must have sufficient efficiency to support real-time analysis without introducing significant latency.

5. Testing and Deployment

The system will be validated in local and lightweight Kubernetes environments such as Minikube and Kind to ensure reproducibility. Functional tests will verify metric ingestion, anomaly flagging, RCA linkage, and UI interactions (filtering, drill-down, timeline sync). Performance tests will stress high-metric-volume scenarios and frequent updates to measure API latency, chart render times, and anomaly detection throughput. Usability tests with operations personnel will evaluate clarity of visualizations, navigation, and the ability to complete troubleshooting tasks quickly. CI scripts will automate linting, unit tests, and integration tests against ephemeral clusters.

Deployment will package backend services as containers, exposed via ingress within the cluster, and the frontend as static assets served through a CDN or Nginx. Configuration will parameterize Prometheus endpoints, Kubernetes API credentials, and model thresholds to adapt to different cluster sizes.

6. Challenges and Limitations

Several practical challenges remain in the scope of this bachelor's design. First, anomaly detection depends on high-quality metrics; noisy or missing data can reduce Isolation Forest accuracy. Second, root cause analysis in this system is exploratory and limited to correlation-based methods; it serves as a decision-support tool where operator judgment remains necessary . Third, UI density must balance information richness with cognitive load to ensure clarity. Finally, integrating heterogeneous log formats can complicate correlation logic, necessitating ongoing refinements.

IV. Conclusion

This research presents the design and implementation of a comprehensive dashboard for Kubernetes cluster management and monitoring, directly addressing the critical operational gaps identified in previous studies. Rather than simply displaying data, this work focuses on transforming raw cluster information into actionable insights.

1. Achievements in Dashboard Design and Integration:

The project successfully developed a unified monitoring platform that merges multiple data sources—including the Kubernetes API, Prometheus metrics, and event logs—into a single, cohesive interface. This integration eliminates "tool sprawl" and provides a holistic view of the cluster, facilitating faster and more accurate decision-making. By implementing a hierarchical information design, the system allows operators to seamlessly drill down from high-level cluster health to granular pod-level details, significantly reducing the cognitive load during system incidents.

2. Intelligent Analysis as a Value Proposition:

The inclusion of an anomaly detection module powered by the **Isolation Forest** algorithm is the cornerstone of this project's transition from a traditional monitoring tool to an intelligent **AIOps** system. By moving away from static, manual threshold-based rules, the system demonstrates the ability to automatically identify abnormal patterns in highly dynamic environments. This approach not only mitigates "alert fatigue" but also provides chronologically ordered evidence that empowers engineers to perform Root Cause Analysis (RCA) with unprecedented speed.

2. Expected Outcomes and Operational Impact:

The deployment of this system in real-world environments is expected to yield measurable improvements, including:

- **Reduced Mean Time to Detection (MTTD):** Through real-time data streaming and automated anomaly scoring.
- **Reduced Mean Time to Resolution (MTTR):** Enabled by the automated correlation between alerts and logs, which accelerates the forensic analysis of failures.
- **Enhanced Operational Efficiency:** Minimizing the manual effort required to monitor and troubleshoot complex distributed systems.

3. Future Outlook:

The microservices-based architecture adopted in this project ensures the system is flexible and scalable. Future work will explore the integration of predictive analytics for capacity planning and the implementation of Natural Language Processing (NLP) to automatically summarize system logs, further advancing the organization's transition toward fully automated, intelligent operations.

In summary, this project demonstrates that combining user-centric design with intelligent data analysis represents the most effective solution for the challenges of managing modern, cloud-native distributed systems.

References

[1] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1), 70-93.

[2] Sharma, P., Chaufournier, L., Shenoy, P., & Tay, Y. C. (2016). Containers and virtual machines at scale: A comparative study. *Proceedings of the 17th International Middleware Conference*, 1-13.

[3] Jaramillo, D., Nguyen, D. V., & Smart, R. (2016). Leveraging microservices architecture by using Docker technology. *2016 IEEE SoutheastCon*, 1-5.

[4] Malygin, D. (2024). Microservice architecture in cloud systems: risks and application opportunities in 2024–2030. *Academia.edu* / Academic Publishing.

[5] Artac, M., Borovssak, T., Di Nitto, E., Guerriero, M., & Tamburri, D. A. (2017). DevOps: Introducing infrastructure-as-code. *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 497-498.

[6] Nedelkoski, S., Cardoso, J., & Kao, O. (2021). Anomaly detection and classification using distributed tracing and deep learning. *Future Generation Computer Systems*, 116, 112-121.

[7] Cerny, T., Donahue, M. J., & Trnka, M. (2018). Contextual understanding of microservice architecture: current and future directions. *Journal of Systems and Software*, 143, 87-106.

[8] Xu, H., Pang, G., Wang, Y., & Wang, Y. (2023). Deep isolation forest for anomaly detection. *IEEE Transactions on Knowledge and Data Engineering*, 35(8), 8208-8221.

[9] Villamizar, M., Garcés, O., Castro, H., Verano, M., Salamanca, L., Kasanka, R., & Merino, F. (2016). Cost comparison of running web applications in the cloud using monolithic, microservice, and serverless architectures. *Journal of Cloud Computing*, 5(1), 14.

[10] Samadbeik, M., Engstrom, T., Lobo, E. H., & Kostner, K. (2024). Healthcare dashboard technologies and data visualization for lipid management: A scoping review. *BMC Medical Informatics and Decision Making*, 24, 33.

[11] Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. *Present and Ulterior Software Engineering*, 195-216.

[12] Hasselbring, W., & Steinacker, G. (2017). Microservice architecture patterns. *Proceedings of the 11th European Conference on Software Architecture (ECSA)*, 221-228.

[13] Wang, Y., Tian, Y., & Wang, L. (2023). Application and development of cloud computing technology. *Information Technology*, 2023.

[14] Zhou, S. (2024). A report on the English-Chinese translation practice of cloud computing for dummies guided by Eugene Nida's functional equivalence theory. *Master's Thesis, Nanjing University of Posts and Telecommunications*.

[15] Wang, Y., Yang, J. H., Xu, C., Ling, X., & Yang, Y. (2015). A review of cloud computing access control technology research. *Journal of Software*, 26(6).

[16] Wu, Z. (2017). Development and trends of cloud computing virtualization technology. *Computer Applications Research*, 34(4).
[17] Jiang, D., Liu, X., & Goh, C. T. (2019). A review of current status of cloud computing t echnology development. *Information Technology*, 2019.
[18] Zhang, L., Lü, J., Yan, F., & Xiong, Y. (2022). A review of trusted cloud computing research. *Journal of Information Security Research*, 8(3).
[19] Li, X., Gao, X., Yan, B., Chen, C., Chen, B., Li, J., & Xu, J. (2019). An anomaly detection method for power dispatch flow data based on the isolated forest algorithm. *Power System Technology*, 43(4).
[20] Song, X. (2023). Research on anomaly target detection method in hyperspectral remote sensing images based on isolation forest algorithm. *Doctoral Thesis, University of Chinese Academy of Sciences*.
[21] Ramírez, T. (2020). FastAPI Framework: high performance, easy to learn, fast to code, ready for production. *Python Software Foundation*.
[22] Fedosejev, A. (2015). React.js Essentials. *Packt Publishing Ltd*.
[23] Wang, V., Salam, S., & Pels, J. (2013). The Definitive Guide to HTML5 WebSocket. *Apress*.

研究思路与研究方法（要求列出论文提纲）

Research Approach:

This project aims to design and implement a web-based dashboard system for Kubernetes cluster monitoring and management, providing functional modules such as cluster overview and resource visualization, workload status monitoring and event tracking, performance metrics analysis and anomaly detection, and safe management operations with RBAC enforcement.

The research will proceed through the following steps: (1) conduct comprehensive literature review on Kubernetes observability, anomaly detection methods, and dashboard architecture patterns; (2) perform requirements analysis by examining existing monitoring tools and identifying gaps in current solutions; (3) design the system architecture including backend API services, frontend user interface, and data integration components; (4) implement the dashboard system with iterative development and testing; (5) evaluate the system through functional testing, performance benchmarking, and usability studies.

Each step will be executed systematically: The literature review will synthesize academic papers and technical documentation to establish theoretical foundations. Requirements analysis will compare existing tools (Kubernetes Dashboard, Lens, Grafana) to identify feature requirements and design constraints. System design will employ microservices architecture patterns with RESTful APIs and responsive web interfaces. Implementation will use Python FastAPI for backend services, React with HTML/CSS/JavaScript for frontend development, and

integration with Kubernetes API and Prometheus for data collection. Evaluation will include unit testing, integration testing, performance profiling, and user task-based usability assessment.

Research Methods:

Literature Research: Conduct systematic review of academic papers on Kubernetes observability, cloud-native monitoring, anomaly detection algorithms (Isolation Forest, LSTM), and dashboard design patterns. Study official Kubernetes documentation, Prometheus query language, and best practices for container monitoring to understand key scientific and technical issues.

Survey and Interview: Analyze requirements through examination of existing monitoring solutions, review user feedback on platforms like GitHub and Stack Overflow, and consult with system administrators to understand pain points in cluster operations. Conduct feasibility analysis considering technical constraints, resource limitations, and deployment scenarios.

Case Study: Perform comparative analysis of existing systems including Kubernetes Dashboard (official), Lens (commercial), Grafana + Prometheus (open-source stack), and Datadog/Dynatrace (enterprise solutions). Evaluate their strengths, limitations, feature sets, and architectural approaches to inform design decisions.

Empirical Research: Evaluate the implemented system through multiple dimensions: (1) Functional evaluation - verify all specified features operate correctly; (2) Performance evaluation - measure API response latencies, page load times, resource consumption, and scalability; (3) Usability evaluation - conduct task-based studies where users diagnose seeded failures and perform management operations; (4) Security evaluation - verify RBAC enforcement and test for privilege escalation vulnerabilities.

Table of Contents

1. Introduction

1.1 Background and Motivation

1.2 Problem Statement: Challenges in Cloud-Native Observability

1.3 Objectives and Research Questions

1.4 Technical Progressiveness and Innovation

1.5 Thesis Structure

2. Literature Review

2.1 Kubernetes Architecture and Observability Fundamentals

2.2 Monitoring and Observability in Cloud-Native Environments

2.2.1 International Research Perspectives

2.2.2 Analysis of Domestic (Chinese) Cloud Computing Research

2.3 Theoretical Basis for Anomaly Detection (Isolation Forest)
2.4 Evolution of AIOps and Intelligent Dashboards
2.5 Comparison of Existing Monitoring Technologies (Prometheus, Grafana, Lens)
3. System Analysis and Design
3.1 Requirements Analysis
3.1.1 Functional Requirements (Data Fusion, Real-time Visualization)
3.1.2 Non-Functional Requirements (Scalability, Latency, Security)
3.2 System Architecture Design
3.2.1 Overall Four-Surface Architecture Design
3.2.2 Backend Service Layer (FastAPI & Asynchronous Processing)
3.2.3 Frontend Presentation Layer (React & WebSocket Integration)
3.3 Module Design and Business Logic
3.3.1 Cluster Health Overview
3.3.2 Deep Resource Monitoring
3.3.3 Intelligent Anomaly Detection and Root Cause Analysis
3.4 Data Flow and Process Modeling (Sequence Diagrams)
4. Implementation of the Kubernetes Dashboard System
4.1 Development Environment and Tech Stack
4.2 Backend Development
4.2.1 Kubernetes API and Client-Go Integration
4.2.2 Prometheus Metric Aggregation and Down-sampling
4.2.3 Machine Learning Implementation: Isolation Forest in Scikit-learn
4.3 Frontend Development
4.3.1 Component-Based UI Design
4.3.2 Real-time Data Streaming via WebSockets and SSE
4.3.3 Visual Analytics using ECharts
4.4 Security: RBAC and Authentication Implementation
5. Results and Discussion
5.1 System Demonstration (Interface Evaluation)
5.2 Functional Validation (Anomaly Detection Accuracy)
5.3 Performance Evaluation
5.3.1 Real-time Update Latency (MTTD Analysis)
5.3.2 Resource Impact on the Host Cluster
5.4 Comparison with Industry Standards (Grafana vs. Proposed System)
5.5 Discussion of Practical Operational Benefits
6. Conclusion
6.1 Summary of Findings
6.2 Key Contributions to Kubernetes Management
6.3 Limitations and Future Research Directions

进程安排

任务	完成时间
完成开题报告	2026 年 01 月 12 日之前
完成论文初稿	2026 年 02 月 28 日之前
完成论文二稿	2026 年 03 月 20 日之前
完成论文定稿	2026 年 04 月 20 日之前

指导老师意见

指导老师签名：

年　　月　　日

专业负责人意见

专业负责人签名：

年　　月　　日

学院审核意见

学院负责人签名：

年　　月　　日